COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	07/10/2026
NAME:	Macasaet, John David Milan, Kherwin Emmanuel Moradillo, Eunice	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

	BEST CASE	WORST CASE	ANALYSIS
Insertion Sort	O(n)	$O(n^2)$	The best case is when the list is already sorted. The program makes n-1 comparisons without making any swaps. The worst case is when the array is in reverse sorted order. The inner loop runs for n(n-1)/2 times.
Bubble Sort	O(n)	$O(n^2)$	If the array is already sorted then it will in the best case time complexity. The inner loop executes n-1 times to check all elements and there is no need to swap any elements resulting in a O(n) time complexity. For the worst case, it is when you need to swap every element to get the complete sorted array. In this case you get $O(n^2)$ because you utilize both the outer loop and inner loop for the whole array with no early termination.

Table 14-1. Best and Worst Case Analysis Using Theoretical Tools

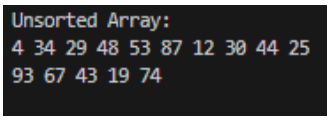
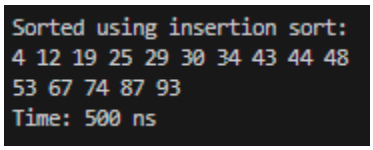
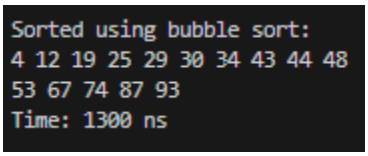
	INSERTION SORT	BUBBLE SORT
		
BEST CASE	The time grew about 10 times per 10 times increase in the n, so it is o(n). This satisfies the theoretical analysis.	In the output, as n grew about 10 times, the time grew about 100 times.
WORST CASE	The time could grow in about 100 times per 10 times increase in the n, therefore it is o(n^2).	As the n increases 10 times, thr time also grew 100 times therefore it is o(n^2).

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

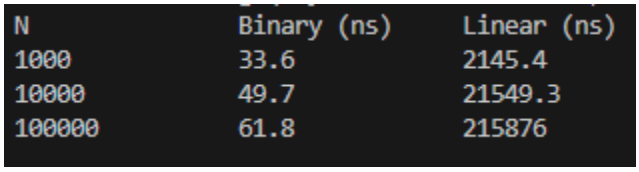
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE	ANALYSIS
Linear Search Step 1: I := 0 Step 2: Repeat while I <= n If (item = data[i]) Print "Searching is successful" Exit Else Print "Searching is Unsuccessful" Exit	For linear search, It checks the elements one by one starting at index 0. The best case is when the key is at the first position. The worst case is when the key is at the last position or not in the list, so it compares all N elements.
Binary Search Step 1: low := 0, up = n-1 Step 2: Repeat while low <= up mid = int(low+up)/2 if (no = arr[mid]) print "Search element is found!" exit	for binary search, It checks the middle element and throws away half of the list each loop. The best case is when the key is the first middle element. The worst case is when the key is not in the list, so the range is halved until it is empty. The list must be sorted.

else if (no < arr[mid]) then up = mid-1 else low = mid+1 Step 3: Print "Search element is not found" Step 4: Exit	

TIME AND SPACE COMPLEXITIES		
	Binary Search	Linear Search
Best Case	O(1)	O(1)
Worst Case	O(N)	O(log N)

Empirical Analysis		
Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1000	33.6 ns	2145.4
10000	49.7 ns	21549.3
100000	61.8 ns	215876



Analysis: Linear search grows about 10 times every time N grows 10 times (2,145 to 21,549 to 215,876 ns). This matches O(N). Binary search barely grows (33.6 to 49.7 to 61.8 ns) even when N becomes 100 times bigger. This matches O(log N), since it only needs about 10, 14, and 17 steps. At N = 100,000, binary search was about 3,500 times faster than linear search. So the experiment agrees with the theory.

CONCLUSION	
MACASAET	In this activity, I learned about the complexities of algorithms and how we can see the runtime and compare algorithms using that. This activity is pretty much a review of previous lessons as

	we pretty much learned about the theoretical and empirical analysis of programs in the past this time we are doing it with different algorithms.
MILAN	In this activity I learned how to compare two algorithms using time and space complexity. I compared linear search and binary search. Linear search has a worst case of O(N), while binary search has a worst case of O(log N). Both use O(1) space.
MORADILLO	In this activity, I easily understood how we should compute the algorithm's time and space complexity. As our professor said, this was a review of our past activities including the sorting algorithm and the analysis. Although the empirical analysis is new to me, we were still able to manage it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.